JavaScript Fundamentals

Crash Course Notes — Roman Urdu

Har section ke saath 3 Coding Practice Tasks

41 Sections · Coding Practice Edition

Table of Contents

1 JavaScript Kya Hai?

- JavaScript ek **high-level, interpreted** language hai.
- **High-level:** memory management jaisi cheezein khud handle nahi karni parti (C/C++ ki tarah low-level nahi hai).
- **Interpreted:** compiler ke through run nahi hota, direct execute hota hai (Java ek compiled language hai, alag cheez hai).
- JavaScript **ECMAScript** specification ko follow karti hai. Doosre implementations bhi hain (JScript, ActionScript) lekin JS sab se popular hai.
- JavaScript **multi-paradigm** hai — object-oriented ya functional, jaisa marzi likh sakte ho.
- Ye browser (client) ki language hai — HTML/CSS ke sath interactivity (form validation, alerts) ke liye use hoti hai.
- **Node.js** ke zariye server-side bhi chal sakti hai (database interaction waghera).
- JS frameworks: React, Angular, Vue.js. Mobile: React Native, NativeScript. Desktop: Electron.

✎ Coding Practice

1. Ek `.js` file banao jisme `console.log()` se apna naam, ek number, aur ek boolean value teen alag statements mein print ho.
2. Ek variable `greeting` banao jiski value `'Hello, World!'` ho aur usko `console.log` karo.
3. Code likho jo `typeof` operator use karke ek string aur ek number ka type console par print kare.

Real-World Use Case

1. Ek script likho jo check kare ke code browser mein chal raha hai ya Node.js mein (`typeof window !== 'undefined'`).
   Hint / Kahan use hota hai: Real use: MERN stack mein wahi utility function frontend aur backend dono mein use hoti hai — ye check environment-specific bugs bachata hai.
2. Ek chhota Node.js script `hello.js` likho jo terminal mein run ho (`node hello.js`) aur ek message print kare.
   Hint / Kahan use hota hai: Real use: Node.js scripts backend automation ke liye use hote hain — jaise seed data generate karna ya cron jobs.

2 Setup & Console

- JS file ko HTML mein link karne ka tareeqa: `<script src="main.js"></script>` — ye body ke end mein, closing `</body>` se pehle likhna chahiye (taake HTML/CSS pehle load ho jaye).
- `alert('text')` ek popup deta hai lekin debugging ke liye achha nahi (script block ho jata hai).
- Console debugging ka best tareeqa hai:
  - `console.log()` — normal output
  - `console.error()` — red error message
  - `console.warn()` — yellow warning message
- Browser dev tools kholna: Chrome mein `Cmd+Option+I` (Mac) ya `F12` (Windows).

✎ Coding Practice

1. Code likho jo pehle `console.log('Starting...')` print kare, phir kisi division ko zero se check kar ke `console.error('Cannot divide by zero')` print kare agar divisor 0 ho.
2. Ek function `checkAge(age)` likho jo agar age 18 se kam ho to `console.warn('Minor')` print kare, warna `console.log('Adult')`.
3. Code likho jo ek array ke 5 elements ko loop se console.log kare, aur agar koi element negative ho to `console.error()` se alag flag kare.

Real-World Use Case

1. Ek function `validateForm(data)` likho jo agar koi field missing ho to `console.error()` se batae kaunsi field missing hai.

   Hint / Kahan use hota hai: Real use: Laravel/Angular forms submit hone se pehle client-side validation errors console mein debug karne ke liye isi tarah log hoti hain.
2. Ek API call simulate karo (fake data) jisme success par `console.log('Data fetched')` aur fail par `console.error('Fetch failed')` ho.

   Hint / Kahan use hota hai: Real use: Har real API integration (jaise CrimeX ka Urdu news data fetch) mein isi pattern se success/failure debug hoti hai.

3 Variables — var, let, const

- **var:** purana tareeqa, globally scoped hota hai — conflicts create kar sakta hai. Ab use nahi karte.
- **let:** ES6 mein aaya — value reassign ki ja sakti hai.
- **const:** value reassign nahi ki ja sakti. Declare karte waqt value dena zaroori hai, warna error aayega.
- General rule: `const` use karo jab tak reassign nahi karna, warna `let` use karo.
- Arrays/objects ke sath `const` use kar sakte ho kyunke unke andar values change ho sakti hain, sirf poori variable ko reassign nahi kar sakte.

✎ Coding Practice

1. `let score = 0;` se shuru karo aur usko 3 dafa increment karke har dafa `console.log` karo (jaise ek game score tracker).
2. `const cart = [];` banao aur `.push()` se 3 items add karo (dikhao ke const array mein bhi mutate ho sakta hai).
3. Code likho jisme `const PI = 3.14;` ko reassign karne ki koshish karo — error ko `try/catch` mein pakar kar console.log karo.

Real-World Use Case

1. Ek e-commerce cart ka `let cartTotal` banao jo har item add hone par update ho (reassign), aur ek `const TAX_RATE = 0.17` banao jo kabhi change na ho.

   Hint / Kahan use hota hai: Real use: Cart totals hamesha changeable (let) hote hain, lekin tax rate jaisi business constants const rakhi jati hain taake accidentally change na ho.

2. Ek subscription platform ke liye `const PLAN_PRICES = { basic: 500, pro: 1500 }` banao — dikhao ke object ki values change ho sakti hain lekin variable reassign nahi ho sakta.

   Hint / Kahan use hota hai: Real use: Tumhare e-learning platform ke subscription plans isi pattern se define hote hain — prices update hote hain lekin structure fix rehta hai.

4 Data Types (Primitive)

- Primitive types: **String**, **Number** (JS mein alag float type nahi hota), **Boolean**, **null** (intentionally empty), **undefined** (value assign nahi hui), **Symbol** (rarely use hota).
- Semicolons JS mein mandatory nahi hain, lekin most developers use karte hain.
- **typeof operator** se data type check karte hain.
- Gotcha: `typeof null` → `"object"` deta hai (legacy JS bug/quirk).

✎ Coding Practice

1. Ek function `getType(value)` likho jo koi bhi value le kar uska data type string ki tarah return kare (`typeof` use karke).
2. 5 variables banao (string, number, boolean, null, undefined) aur ek loop/array se sab ka `typeof` ek sath print karo.
3. Ek function likho jo do values le kar check kare ke unka `typeof` same hai ya nahi, aur boolean return kare.

Real-World Use Case

1. Ek function `isValidInput(value)` likho jo form input ke liye check kare ke value `undefined` ya `null` to nahi (login form scenario).

   Hint / Kahan use hota hai: Real use: Angular/React form validation mein isi tarah undefined/ null inputs ko submit hone se pehle block kiya jata hai.

2. Ek user object banao jisme `email` field ho aur agar wo empty string ho to usko falsy treat kar ke error dikhane wala function likho.

   Hint / Kahan use hota hai: Real use: Signup forms mein email field empty check karna sabse common validation hai — Laravel backend bhi isi tarah null/empty check karta hai.

5 Strings

- Concatenation (old way): `'My name is ' + name + ' and I am ' + age`
- Template literals (ES6, better way): backticks (`` ` ``) use karke `${variableName}` syntax.
- Common string properties/methods: `.length`, `.toUpperCase()` / `.toLowerCase()`, `.substring(start, end)`, `.split(separator)`.
- Methods ko chain kiya ja sakta hai, e.g. `.substring(0,5).toUpperCase()`

✎ Coding Practice

1. Ek function `getInitials(firstName, lastName)` likho jo dono naamon ke pehle letters uppercase mein jodh kar return kare (e.g. 'Ali Khan' → 'AK').
2. Ek function `isPalindrome(str)` likho jo check kare ke string ulti taraf se bhi wahi padhti hai ya nahi (e.g. 'madam' → true).
3. Ek function `countVowels(str)` likho jo diye gaye string mein vowels (a,e,i,o,u) ki total count return kare.

Real-World Use Case

1. Ek function `generateSlug(title)` likho jo blog title ko lowercase kar ke spaces ko hyphen (`-`) se replace kare (e.g. 'My First Blog' → 'my-first-blog').

   Hint / Kahan use hota hai: Real use: Blogging platforms (jaise Medium) URL slugs isi tarah generate karte hain — SEO-friendly URLs banane ke liye.

2. Ek function `maskEmail(email)` likho jo email ka username part partially hide kare (e.g. 'ali@gmail.com' → 'a**@gmail.com').

   Hint / Kahan use hota hai: Real use: Payment/banking apps mein sensitive info (email, card number) isi tarah masked dikhayi jati hai privacy ke liye.

6 Arrays

- Array banane ke do tareeqe: Constructor `new Array(1,2,3)`, Literal `const fruits = ['apple','orange','pear']` (common way).
- JS arrays mein alag-alag data types ek sath rakhe ja sakte hain.
- Zero-based indexing. Access: `fruits[1]`
- Common methods: `.push()`, `.unshift()`, `.pop()`, `Array.isArray()`, `.indexOf()`.
- Higher order methods (loops se prefer): `.forEach()`, `.map()`, `.filter()` — chain ki ja sakti hain.

✎ Coding Practice

1. Ek function `doubleEvens(arr)` likho jo `.filter()` aur `.map()` chain kar ke sirf even numbers ko double kar ke naya array return kare.
2. Ek function `findMax(arr)` likho (bina `Math.max` ke) jo array mein sabse bara number loop se dhoond kar return kare.
3. Ek function `removeDuplicates(arr)` likho jo array se duplicate values hata kar naya array return kare.

Real-World Use Case

1. Ek function `getCartTotal(prices)` likho jo cart ke prices array ka sum `.reduce()` ya loop se nikale.

   Hint / Kahan use hota hai: Real use: Har e-commerce checkout page (Amazon, Daraz) yehi calculation cart total ke liye karta hai.

2. Ek function `filterInStock(products)` likho jo products array (har object mein `inStock: true/false`) se sirf available items `.filter()` kare.

   Hint / Kahan use hota hai: Real use: Product listing pages sirf in-stock items dikhane ke liye isi filter logic ko backend query ya frontend filter mein use karte hain.

7 Objects (Object Literals)

- Key-value pairs: `const person = { firstName: 'John', lastName: 'Doe', age: 30 }`
- Objects ke andar arrays aur nested objects bhi ho sakte hain.
- Access: dot notation `person.firstName`. Nested: `person.address.city`
- Destructuring (ES6): `const { firstName, lastName } = person;` Nested: `const { address: { city } } = person;`
- Property add: `person.email = '...'`. Array of objects common pattern hai.

✎ Coding Practice

1. Ek `book` object banao (title, author, pages) aur ek function `describeBook(book)` likho jo destructuring se ek sentence string return kare.
2. Ek `student` object banao jisme nested `address: {city, country}` ho, aur destructuring se sirf `city` nikal kar print karo.
3. Ek array of `students` objects (name, marks) banao aur `.filter()` se sirf pass (marks >= 50) students ka array return karne wala function likho.

Real-World Use Case

1. Ek `order` object banao (orderId, items array, totalAmount, status) jaisa ke ek real API response hota hai, aur destructuring se `status` nikal kar print karo.

   Hint / Kahan use hota hai: Real use: REST APIs se aane wale JSON responses ko exactly isi tarah destructure kar ke frontend components mein use kiya jata hai.

2. Ek `donor` object banao (name, amount, date) apne charity fund module jaisa, aur ek function likho jo receipt string generate kare.

   Hint / Kahan use hota hai: Real use: Tumhare e-learning platform ke charity/disbursement module mein donor records isi structure mein store hote hain.

8 JSON

- JSON — data format hai, APIs/backend ke sath data bhejne-lene ke liye use hota hai.
- Farq object literal se: saari keys aur strings double quotes mein honi chahiye.
- `JSON.stringify(object)` — JS object ko JSON string mein convert karta hai.

✎ Coding Practice

1. Ek `user` object banao, usko `JSON.stringify()` se string mein convert karo, aur uski `typeof` print karo (dikhao ab string hai).
2. Ek function `toJSON(obj)` likho jo koi bhi object le kar uska JSON string return kare.
3. Ek array of objects banao (products: name, price) aur poore array ko ek JSON string mein convert kar ke console.log karo.

Real-World Use Case

1. Ek `studentProfile` object ko `JSON.stringify()` se convert karo jaisa ke ek POST request body backend ko bhejta hai.

   Hint / Kahan use hota hai: Real use: Angular ka `HttpClient` jab Laravel backend ko POST request bhejta hai, to body automatically JSON string ki tarah jaati hai.

2. Ek fake API response string (JSON format mein, single-quote galtiyon ke sath) likho aur explain karo ye kyun backend reject karega.

   Hint / Kahan use hota hai: Real use: Backend APIs (Laravel/Express) strict JSON format expect karte hain — invalid JSON se 400 Bad Request error milta hai.

9 Loops

- `for` loop: initialization, condition, increment — teen parts.
- `while` loop: variable bahar declare, phir condition check.
- `for...of` : arrays loop karne ka readable tareeqa.
- Higher order array methods generally for/while se better hain array iteration ke liye.

✎ Coding Practice

1. `for` loop se 1 se 100 tak sab numbers ka sum calculate karo aur print karo.
2. `while` loop use karke ek function `countDigits(num)` likho jo kisi number ke digits ki ginti return kare.
3. `for...of` loop se ek array mein saare strings ki total combined length calculate karo.

Real-World Use Case

1. Ek loop likho jo 10 fake 'users' ke liye IDs generate kare (user_1 se user_10 tak) — jaisa seed data banate waqt hota hai.

   Hint / Kahan use hota hai: Real use: Database seeding (test data banate waqt) mein loops se dummy records generate kiye jate hain.

2. Ek loop likho jo pagination simulate kare — 100 records ko 10-10 ke groups (pages) mein console.log kare.

   Hint / Kahan use hota hai: Real use: Har listing page (products, blogs, users) pagination isi logic se implement hoti hai backend query ke sath.

10 Conditionals

- `if/else if/else` — normal conditional structure.
- Comparison: `==` (sirf value), `===` (value + type, recommended).
- Logical: `||` (OR), `&&` (AND).
- Ternary: `const color = x > 10 ? 'red' : 'blue';`
- `switch` statement — har case ke baad `break` zaroori.

✎ Coding Practice

1. Ek function `getGrade(marks)` likho jo if/else if/else se A/B/C/F grade return kare (marks ke basis par).
2. Ek function `fizzBuzz(n)` likho jo 1 se n tak loop kare — multiples of 3 par 'Fizz', multiples of 5 par 'Buzz', dono par 'FizzBuzz', warna number print kare.
3. Ek `switch` statement wala function `getDayName(num)` likho jo number (1-7) ke basis par din ka naam return kare (default case ke sath 'Invalid').

Real-World Use Case

1. Ek function `canAccessCourse(user)` likho jo check kare ke user ka subscription active hai aur payment verified hai (dono conditions `&&` se).

   Hint / Kahan use hota hai: Real use: Tumhare subscription-based e-learning platform mein isi logic se decide hota hai ke student course content access kar sakta hai ya nahi.

2. Ek ternary se ek `orderStatusColor` variable banao jo status ke basis par 'green' ya 'red' ho (delivered vs cancelled).

   Hint / Kahan use hota hai: Real use: Order tracking UI (jaise Daraz app) mein status badges isi tarah conditionally color hote hain.

11 Functions

- Normal function: `function addNums(num1, num2) { return num1 + num2; }`
- Default parameters: `function addNums(num1 = 1, num2 = 1) {}`
- Arrow functions (ES6): `const addNums = (num1, num2) => { return num1 + num2; }`
- Single expression: curly braces aur return drop kar sakte hain. Sirf ek parameter: parenthesis bhi drop.
- Arrow functions ka lexical `this` hota hai (advanced topic, section 34/36 mein detail).

✎ Coding Practice

1. Ek arrow function `isEven(num)` likho (one-liner) jo boolean return kare.
2. Ek function `calculateArea(shape, ...dims)` likho jo default parameters ke sath rectangle/circle ka area calculate kare.
3. Ek arrow function `square = n => n * n` likho aur usko ek array par `.map()` ke sath use karo.

Real-World Use Case

1. Ek function `calculateDiscount(price, discountPercent = 10)` likho jo default 10% discount apply kare agar percent na diya jaye.

   Hint / Kahan use hota hai: Real use: E-commerce sales/promo code systems mein default discount values isi tarah handle hoti hain.

2. Ek arrow function `formatCurrency = (amount) => \`Rs. ${amount.toFixed(2)}\`` likho aur prices array par `.map()` ke sath use karo.

   Hint / Kahan use hota hai: Real use: Currency formatting utility functions har frontend app mein reusable helper ki tarah use hote hain.

12 Object-Oriented Programming (OOP) — Constructors & Classes

- Constructor Functions (ES5): `function Person(firstName, lastName, dob) { this.firstName = firstName; ... }`
- Instantiate: `const person1 = new Person(...)`
- Prototype: `Person.prototype.getFullName = function() {...}` — memory-efficient (har instance mein duplicate nahi hota).
- ES6 Classes — syntactic sugar peeche se prototypes hi use karti hain.

✎ Coding Practice

1. Ek `Car` constructor function banao (make, model, year) aur prototype par `getInfo()` method add karo jo ek sentence return kare.
2. Wahi `Car` ko ES6 `class` syntax se dobara likho, aur 3 instances bana kar sab ka `getInfo()` print karo.
3. Ek `BankAccount` class banao jisme `balance` property ho aur `deposit(amount)` / `withdraw(amount)` methods hon.

Real-World Use Case

1. Ek `Student` class banao (name, enrolledCourses array) jisme `enroll(courseName)` method ho jo array mein course add kare.

   Hint / Kahan use hota hai: Real use: Tumhare e-learning platform ke Angular frontend mein Student model isi tarah design hota hai.

2. Ek `DonationRecord` class banao (donorName, amount, date) jisme `getReceiptText()` method ho.

   Hint / Kahan use hota hai: Real use: Charity/recipient fund module mein disbursement tracking records isi class-based structure se manage hote hain.

13 DOM (Document Object Model)

- `window` object top-level parent object (alert, localStorage, fetch). `document` object DOM ko represent karta hai.
- Single selectors: `getElementById`, `querySelector`. Multiple: `querySelectorAll` (NodeList), `getElementsByClassName` / `getElementsByTagName` (HTMLCollection).
- Manipulation: `.remove()`, `.textContent` / `.innerText`, `.innerHTML`, `.style.propertyName`, `.classLi` `remove`.
- Events: `addEventListener('event', callback)`, `e.preventDefault()`, `e.target`.

✎ Coding Practice

1. HTML page banao jisme ek button ho — click hone par `querySelector` se select kar ke us button ka background color `.style.background` se badle.
2. `querySelectorAll` se saare `<li>` select karo aur `.forEach()` se har ek par click listener lagao jo click hone par us item ko `.remove()` kar de.
3. Ek counter app banao: ek `<p>` mein number 0 dikhaye, do buttons (+ aur −) us number ko `.textContent` se update karein.

Real-World Use Case

1. Ek simple login form banao (email, password inputs) — submit par dono fields ki values `querySelector` se nikal kar console.log karo.

   Hint / Kahan use hota hai: Real use: Har login page pehle frontend par values collect karti hai, phir backend API ko bhejti hai.

2. Ek 'like' button banao jo click hone par apna text 'Liked ' mein badle aur `.classList.add('liked')` kare.

   Hint / Kahan use hota hai: Real use: Social media/blog platforms (jaise Medium) mein like buttons isi DOM manipulation pattern se kaam karte hain.

14 Mini Project Pattern (Form + List)

- DOM elements ko variables mein store karna (form, inputs, message div, list container).
- Submit event listener + `preventDefault()`. Validation: `input.value === ''` check.
- `setTimeout(callback, ms)` — delay ke baad execute/remove.
- `document.createElement('li')` + `.appendChild(document.createTextNode('text'))` + `parentElement.appendChild(newElement)`.

✎ Coding Practice

1. Ek to-do list app banao: form se input lo, submit par empty-check validation karo, aur valid input ko list mein naye `<li>` ki tarah add karo.
2. To-do list mein har `<li>` ke sath ek 'delete' button add karo jo click hone par wo item list se hata de.
3. Agar input empty ho to ek error message dikhao jo `setTimeout` se 3 second baad khud DOM se remove ho jaye.

Real-World Use Case

1. Ek 'Add Student' mini form banao jo naam input le kar ek list mein add kare, aur duplicate naam par error dikhaye.

   Hint / Kahan use hota hai: Real use: Admin dashboards (jaise tumhare e-learning platform ka admin panel) mein isi pattern se records add hote hain UI se.

2. Comment box jaisa form banao jisme submit ke baad naya comment list ke top par add ho (end ki bajaye).

   Hint / Kahan use hota hai: Real use: Blog/course discussion sections mein naye comments hamesha sabse upar dikhaye jate hain.

15 Beyond This Crash Course

- Data persist karne ke liye: Backend + database (Node.js, PHP, Python) + fetch API/Ajax.
- `localStorage` — browser mein hi data store karna (sirf us user ke browser tak mehdood).

✎ Coding Practice

1. Code likho jo ek `name` variable ko `localStorage.setItem('name', name)` se save kare, phir `localStorage.getItem('name')` se wapas nikal kar print kare.
2. Section 14 wali to-do list ko modify karo taake naye items `localStorage` mein bhi save hon (JSON.stringify ke sath).
3. Code likho jo page load hone par `localStorage` se saved to-do items nikal kar list mein dikhaye (JSON.parse ke sath).

Real-World Use Case

1. Ek 'dark mode' toggle button banao jiski state (on/off) `localStorage` mein save ho aur page reload par bhi yaad rahe.
   Hint / Kahan use hota hai: Real use: Har modern web app (including tumhari dark-themed PDF pipeline jaisi UI) mein theme preference isi tarah persist hoti hai.
2. Ek 'recently viewed courses' feature simulate karo — course IDs ko `localStorage` array mein store karo jab user click kare.
   Hint / Kahan use hota hai: Real use: E-learning/e-commerce platforms 'recently viewed' feature isi client-side storage se banate hain bina backend call ke.

16 Objects — Deep Dive

- Real-life object ki tarah, JS object ke properties aur methods hote hain.
- Object literal notation curly braces `{}` se banate hain.
- Access: dot notation (`user.name`) ya square bracket (`user['name']` — dynamic keys ke liye zaroori).
- Methods add karna: key ki value ek function. Shorthand syntax (ES6): `login() {...}`
- **this keyword:** method ke andar object ki property access karne ke liye — JS this ki value automatically us object pe set kar deta hai jispar method call hua.
- Arrow function methods ke andar `this` ko bind nahi karti — regular function use karo.

✎ Coding Practice

1. Ek `calculator` object banao jisme `add()` , `subtract()` , `multiply()` methods hon jo `this.result` update karein.
2. Ek function likho jo dynamic key access se kisi object ki value nikale: `getProperty(obj, keyName)` jo `obj[keyName]` return kare.
3. Ek object banao jisme ek method `greet()` ho — usko pehle regular function se aur phir arrow function se likh kar dikhao ke `this.name` ka output kaise alag aata hai.

Real-World Use Case

1. Ek `shoppingCart` object banao jisme `addItem()` , `removeItem()` , aur `getTotal()` methods hon jo `this.items` array manage karein.

   Hint / Kahan use hota hai: Real use: Cart logic (chahe vanilla JS ho ya React context) isi object-based pattern se design hoti hai.

2. Ek `videoPlayer` object banao (currentTime, duration) jisme `play()` , `pause()` methods `this` use karein.

   Hint / Kahan use hota hai: Real use: Course video players (jaise Udemy) internally isi tarah state object maintain karte hain.

17 Array of Objects

- Agar array ke har element ke paas multiple properties honi chahiye, to har element ko object banate hain.
- Real-world data (jaise database se aane wala data) usually array of objects format mein hota hai.
- Loop karte waqt har item object hoti hai, is liye uski properties access ki ja sakti hain.

✎ Coding Practice

1. Ek array of `employees` objects (name, salary) banao aur ek function likho jo sabki total salary `.reduce()` ya loop se calculate kare.
2. Us array mein se ek function `getHighestPaid(employees)` likho jo sabse zyada salary wale employee ka naam return kare.
3. `.map()` se employees array se sirf ek naya array banao jisme har object mein `{name, bonus: salary * 0.1}` ho.

Real-World Use Case

1. Ek array of `notifications` objects (message, isRead) banao aur function likho jo unread count `.filter()` se nikale.

   Hint / Kahan use hota hai: Real use: Har app ka notification bell icon (unread badge count) isi logic se calculate hota hai.

2. Ek array of `transactions` objects (amount, type: 'credit'/'debit') banao aur total balance calculate karne wala function likho.

   Hint / Kahan use hota hai: Real use: Charity fund module ka disbursement tracking/ transparency report isi tarah transactions ko process karta hai.

18 Math Object (Built-in Object)

- Properties (constants): `Math.PI`, `Math.E`.
- Methods: `Math.round()`, `Math.floor()`, `Math.ceil()`, `Math.trunc()`, `Math.random()`.
- 1 se 100 ke beech random number: `Math.round(Math.random() * 100)`

✎ Coding Practice

1. Ek function `getRandomInt(min, max)` likho jo min aur max ke beech (dono inclusive) ek random integer return kare.
2. Ek function `circleArea(radius)` likho jo `Math.PI` use kar ke area calculate kare, aur `Math.round()` se 2 decimal points tak round kare.
3. Ek 'dice roll' function `rollDice()` likho jo 1 se 6 tak random number return kare, aur usko 10 dafa call kar ke results array mein store karo.

Real-World Use Case

1. Ek function `generateOTP()` likho jo `Math.random()` se 6-digit random OTP code generate kare.
   Hint / Kahan use hota hai: Real use: Login/signup verification (SMS/email OTP) systems isi tarah random codes generate karte hain.
2. Ek function `generateOrderId()` likho jo `Math.floor(Math.random() * 1000000)` se ek unique-jaisi order ID banaye.
   Hint / Kahan use hota hai: Real use: E-commerce order tracking IDs (temporary/demo purposes ke liye) isi tarah random generate kiye ja sakte hain.

19 Primitive vs Reference Types

- 7 data types, 6 primitive (number, string, boolean, null, undefined, symbol). Reference: object (arrays, functions, dates waghera).
- Primitives → stack memory. Reference types → heap memory, stack par sirf pointer.
- Primitive copy: nayi, alag value banati hai. Reference copy: sirf pointer copy hota hai, same object share hota hai.

✎ Coding Practice

1. Code likho: `let a = 10; let b = a; b = 20;` — phir dono ko print kar ke dikhao ke `a` affect nahi hota.
2. Code likho: `let obj1 = {x: 1}; let obj2 = obj1; obj2.x = 99;` — phir dono print kar ke dikhao ke `obj1.x` bhi change ho jata hai.
3. Ek function `cloneObject(obj)` likho (spread syntax `{...obj}` use kar ke) jo ek independent copy banaye — test karo ke copy change karne se original change nahi hota.

Real-World Use Case

1. Ek function `updateUserSettings(settings)` likho jo settings object ki copy (spread se) banaye aur update kare — dikhao original object change nahi hota.

   Hint / Kahan use hota hai: Real use: React/Angular state management mein hamesha immutable updates ki jati hain (original state directly mutate nahi karte).

2. Ek cart array ko naye variable mein assign kar ke ek item change karo — dikhao dono variables affect hote hain (phir spread se fix karo).

   Hint / Kahan use hota hai: Real use: Ye ek common React bug hai (state directly mutate karna) — spread operator se copy bana kar isko avoid kiya jata hai.

20 Object-Oriented Programming (OOP) — Fundamentals

- OOP ek programming paradigm hai. OOP se pehle procedural programming hoti thi ("spaghetti code" ka masla).
- OOP mein related properties aur methods ko ek object mein group karte hain.
- **Four Core Pillars:** Encapsulation (grouping), Abstraction (hide complexity), Inheritance (redundant code khatam), Polymorphism (many forms, switch/case avoid).

✎ Coding Practice

1. Ek `Animal` class banao jisme `speak()` method ho, phir ek `Dog` class jo `extends Animal` kare aur `speak()` ko override kare (Inheritance + Polymorphism).
2. Ek `Shape` base class aur do child classes (`Circle`, `Square`) banao — har ek ka apna `area()` method ho jo polymorphism dikhaye.
3. Ek `Wallet` class banao jisme `balance` private-jaisa rakho (naming convention `#balance` ya closure se) aur sirf `addMoney()` / `getBalance()` methods se access ho (Encapsulation).

Real-World Use Case

1. Ek `PaymentMethod` base class banao aur `CreditCard`, `EasyPaisa` child classes banao jo apna `processPayment()` implement karein (Polymorphism).

   Hint / Kahan use hota hai: Real use: Payment gateways (Stripe, JazzCash, EasyPaisa) integration mein isi tarah alag payment methods ek common interface follow karte hain.
2. Ek `NotificationService` class banao jisme `send(message)` ho — internal logic (kaise bheja) hide rahe, sirf ye method public ho.

   Hint / Kahan use hota hai: Real use: Abstraction ki wajah se agar kal ko SMS provider change karein (Twilio se kisi aur mein), baaki app ka code change nahi karna parta.

21 Objects Banane Ke Tareeqe (Factory vs Constructor)

- Object Literal ka masla: methods duplicate karna parta hai agar same jaisa doosra object chahiye.
- Factory Function: regular function jo object return karta hai.
- Constructor Function: `this` + `new` operator use karta hai. Capital letter naming convention.
- **new operator ke peeche 3 cheezein:** naya empty object create, `this` us object ko point, object automatically return.
- `constructor` property — har object mein hoti hai, us function ko reference karti hai jisne object banaya.

✎ Coding Practice

1. Ek `createUser(name, email)` factory function likho jo `{name, email, isActive: true}` object return kare, aur usko 3 dafa call kar ke 3 users banao.
2. Wahi `User` ko ek constructor function se banao (`this` + `new` ke sath), aur prototype par `deactivate()` method add karo.
3. Code likho jo `obj.constructor.name` print kare kisi object literal aur kisi custom constructor se banaye gaye object dono ke liye — output compare karo.

Real-World Use Case

1. Ek `createNotification(type, message)` factory function likho jo type ke basis par (error/success/info) alag styled object return kare.
   Hint / Kahan use hota hai: Real use: Toast notification libraries (react-toastify jaisi) internally isi factory pattern se notification objects banati hain.
2. Ek `Course` constructor function banao (title, instructor, price) aur usse 3 courses banao jaisa ek course catalog hota hai.
   Hint / Kahan use hota hai: Real use: Tumhare e-learning platform ka course catalog data isi structure mein backend se aata hai aur frontend models banate hain.

22 Functions Bhi Objects Hain

- Functions bhi objects hote hain — apni properties/methods hoti hain (`.name` , `.length`).
- `.call(context, arg1, arg2, ...)` — function call karta hai, pehla argument `this` set karta hai.
- `.apply(context, [args])` — `.call()` jaisa, arguments array ke through.
- `new` operator internally `.call()` jaisa mechanism use karta hai.

✎ Coding Practice

1. Ek function `introduce(city, country)` likho jo `this.name` use kare, phir usko `.call()` se do alag objects ke context mein call karo.
2. Wahi function ko `.apply()` se call karo (arguments ko array mein pass kar ke) aur output `.call()` wale se compare karo.
3. Ek function banao aur uska `.name` aur `.length` property print kar ke dikhao (function object hone ka proof).

Real-World Use Case

1. Ek generic `logAction(action)` function banao jo `this.userName` use kare, phir usko `.call()` se alag-alag logged-in users ke context mein use karo.

   Hint / Kahan use hota hai: Real use: Activity logging systems (audit trail) mein isi tarah ek hi log function alag users ke context mein reuse hota hai.

2. Ek utility function ko `.apply()` se call karo jisme arguments ek dynamic array (jaise form fields ki values) se aa rahe hon.

   Hint / Kahan use hota hai: Real use: Dynamic form validation functions jinke arguments count fix nahi hota, unko isi tarah apply se call kiya jata hai.

23 Primitive vs Reference — Function Arguments Ke Sath

- Function ko primitive pass karne par value copy hoti hai — dono independent.
- Function ko object pass karne par reference/pointer copy hota hai — original bhi change hota hai.

✎ Coding Practice

1. Ek function `addTen(num)` likho jo parameter mein 10 add kare — call karne ke baad original variable print kar ke dikhao ke wo change nahi hota.
2. Ek function `addProperty(obj)` likho jo `obj.newProp = true` set kare — call karne ke baad original object print kar ke dikhao ke wo change ho jata hai.
3. Ek function `resetArray(arr)` likho jo `arr.length = 0` kare — dikhao ke original array (reference type) bhi empty ho jata hai.

Real-World Use Case

1. Ek function `applyDiscount(cartItem)` likho jo object parameter ki `price` property directly modify kare — dikhao original cart item bhi change hota hai.

   Hint / Kahan use hota hai: Real use: Ye ek common bug source hai cart logic mein — object mutation ki wajah se UI mein unexpected changes aa sakte hain.

2. Ek function likho jo student ka `marks` number pass kar ke usme grace marks add kare — dikhao original marks variable change nahi hota (primitive).

   Hint / Kahan use hota hai: Real use: Grading systems mein calculations karte waqt original score safe rehna chahiye — isi wajah se primitives return karke naya variable banate hain.

24 Object Properties — Dynamic Nature

- JS objects dynamic hote hain — create karne ke baad bhi properties add/delete ki ja sakti hain.
- Property add: `obj.newProp = value;`. Delete: `delete obj.propName;`
- `for...in` loop — object ki saari keys par loop karta hai. `Object.keys(obj)` — keys ka array.
- `in` operator — check karta hai ke property exist karti hai ya nahi.

✎ Coding Practice

1. Ek function `addTimestamp(obj)` likho jo kisi bhi object mein runtime par `createdAt: Date.now()` property add kare.
2. Ek function `objectToArray(obj)` likho jo `for...in` se object ke saare key-value pairs ko `[key, value]` arrays ke array mein convert kare.
3. Ek function `hasProperty(obj, key)` likho jo `in` operator use kar ke boolean return kare.

Real-World Use Case

1. Ek `user` object banao aur login hone ke baad runtime par usme `authToken` property add karo (jaisa JWT token milta hai).

   Hint / Kahan use hota hai: Real use: Login API response ke baad frontend user object mein token isi tarah dynamically attach karta hai.

2. Logout hone par `user` object se `authToken` property `delete` karo.

   Hint / Kahan use hota hai: Real use: Logout functionality mein session/token data isi tarah clear kiya jata hai frontend state se.

25 Abstraction Implement Karna — Private Members & Closures

- Property ko hide karne ke liye constructor function ke andar sirf local variable define karte hain (object ki property nahi banate).
- **Closure:** inner function apne parent function ke variables "remember" kar leta hai, chahe parent function pehle hi execute ho chuka ho.
- **Getters/Setters** (`Object.defineProperty`) — `get` read ke liye, `set` validation ke sath update ke liye.

✎ Coding Practice

1. Ek `createCounter()` function likho jo closure use kar ke ek private `count` variable rakhe aur `{increment(), decrement(), getCount()}` return kare.
2. Ek `BankAccount` constructor likho jisme `balance` private variable ho (closure se), aur sirf `deposit()` / `getBalance()` se access ho.
3. `Object.defineProperty()` se ek object banao jisme `age` property ka setter negative values reject kare (`throw new Error` ke sath).

Real-World Use Case

1. Ek `createRateLimiter(maxRequests)` function banao jo closure se ek private `requestCount` rakhe aur `allowRequest()` function return kare.

   Hint / Kahan use hota hai: Real use: API rate limiting (jaise 'sirf 5 requests per minute') isi closure pattern se implement hoti hai.
2. Ek `createShoppingCart()` function banao jisme `items` private ho (closure se), sirf `addItem()` / `getItems()` se access ho.

   Hint / Kahan use hota hai: Real use: State ko accidental external modification se bachane ke liye closures / private state pattern professional apps mein use hota hai.

26 Variable Naming Rules (Detailed)

- Sirf letters, numbers, underscore, dollar sign allowed. Number se start nahi ho sakta.
- Reserved keywords use nahi ho sakte. Variables case-sensitive hote hain.
- Naming conventions: **camelCase** (JS standard), **snake_case**.
- Descriptive names use karo, single letters ya overly long names avoid karo.

✎ Coding Practice

1. Ek chhota program likho (5-6 variables) jisme user ki basic info (naam, umar, city) store ho — sab variables camelCase mein descriptive naamon ke sath.
2. Ek function `isValidVariableName(name)` likho jo regex ya loop se check kare ke diya gaya string valid JS variable name hai ya nahi.
3. Ek function `toCamelCase(snakeStr)` likho jo `'user_first_name'` jaisi snake_case string ko `'userFirstName'` camelCase mein convert kare.

Real-World Use Case

1. Ek real signup form ke liye variables banao: `firstName`, `lastName`, `confirmPassword`, `isTermsAccepted` — sab camelCase mein.
   Hint / Kahan use hota hai: Real use: Angular/React form state variables isi naming convention se team-wide consistency maintain karte hain.
2. Ek badly named variable set (`a`, `b`, `flag1`) ko ek real checkout form context mein descriptive names mein rewrite karo.
   Hint / Kahan use hota hai: Real use: Code review mein sabse zyada comments hi bad variable naming par aate hain — descriptive naam maintainability barhate hain.

27 Scope — Global, Function, Aur Block

- Global scope: function/block ke bahar declare, har jagah available.
- Local scope: Function scope, Block scope (curly braces `{}`).
- Values sirf child se parent ki taraf mil sakti hain, reverse nahi.
- `var` function-scoped hai (block ignore karta hai). `let` / `const` block-scoped hain.
- `var` redeclaration allowed (bug-prone). `let` / `const` error dete hain.

✎ Coding Practice

1. Code likho jisme `if` block ke andar `var x = 5;` ho — block ke bahar `x` print kar ke dikhao ke accessible hai.
2. Wahi code `let` se dobara likho — dikhao ke block ke bahar `x` access karne par error aata hai (try/catch mein pakro).
3. Ek function likho jisme ek global aur ek local (function-scoped) variable ho — dikhao console.log se ke function ke andar dono accessible hain.

Real-World Use Case

1. Ek `processOrder()` function banao jisme block-scoped `let discount` ho jo sirf ek `if` condition ke andar zaroori ho — dikhao bahar accessible nahi.

   Hint / Kahan use hota hai: Real use: Scope discipline se bugs kam hote hain — variables sirf wahan available hote hain jahan zaroorat hai.

2. Ek loop ke andar `let` se counter banao aur dikhao har iteration mein naya scope milta hai (jaise setTimeout ke andar closures mein important hota hai).

   Hint / Kahan use hota hai: Real use: Ye classic interview/real bug hai — `var` se loop mein setTimeout galat value capture karta hai, `let` se sahi.

28 Loops — Detailed (for, while, do-while, for-in, for-of)

- 5 tarah ke loops: `for`, `while`, `do-while`, `for-in`, `for-of`.
- `for`: reverse order mein bhi chala sakte hain. Odd/even filter: `if (i % 2 === 0)`
- `while`: increment bhoolna infinite loop bana sakta hai.
- `do-while`: block kam se kam ek baar zaroor chalta hai (condition baad mein check hoti hai).
- `for...in`: object keys ke liye. `for...of`: array values ke liye.

✎ Coding Practice

1. Ek function `factorial(n)` likho `for` loop use kar ke jo n! calculate kare.
2. Ek function `reverseNumber(n)` likho `while` loop use kar ke jo number ke digits ulta kar ke return kare (e.g. 123 → 321).
3. Ek object `settings` banao (3-4 keys) aur `for...in` se uski har key aur value print karo.

Real-World Use Case

1. Ek function `generateInvoiceRows(items)` likho jo `for` loop se har item ka ek invoice line string banaye.

   Hint / Kahan use hota hai: Real use: PDF invoice generation (jaisa tumhara WeasyPrint pipeline) mein rows isi tarah loop se build hoti hain.

2. Ek function `retryRequest(maxAttempts)` likho jo `do-while` se kam se kam ek dafa API call attempt kare, fail hone par retry kare.

   Hint / Kahan use hota hai: Real use: Network requests mein retry logic (jaise fetch fail hone par dobara try) isi pattern se implement hoti hai.

29 Primitive Data Types — Immutability (MDN)

- 7 primitive types: string, number, bigint, boolean, undefined, symbol, null.
- Primitives immutable hote hain — value khud mutate nahi hoti, variable reassign hota hai.
- **Auto-boxing:** `"hello".toUpperCase()` call karte waqt JS temporary wrapper object banata hai.

✎ Coding Practice

1. Code likho: `let str = 'hello'; str.customProp = 1;` `console.log(str.customProp);` — result print kar ke observe karo (undefined aayega).
2. Ek function likho jo ek string ko parameter le kar uska uppercase version return kare, aur dikhao original string variable change nahi hota.
3. 3 auto-boxing examples likho: ek primitive string par `.length`, `.charAt(0)`, aur `.slice(1,3)` call kar ke results print karo.

Real-World Use Case

1. Ek function `sanitizeInput(str)` likho jo user input string ko trim aur lowercase kare — dikhao original input variable change nahi hota.

   Hint / Kahan use hota hai: Real use: Form input sanitization (backend bhejne se pehle) mein hamesha naya cleaned string banaya jata hai, original untouched rehta hai.

2. Ek search feature simulate karo jisme user ka `searchQuery` string `.toLowerCase().trim()` se process ho case-insensitive matching ke liye.

   Hint / Kahan use hota hai: Real use: Har search bar (jaise course search) case-insensitive matching ke liye isi pattern se query process karta hai.

30 Loops — Aur Deep (break, continue, labeled statements)

- `break` : loop turant terminate. `continue` : current iteration skip, agli se continue.
- Labeled statements: nested loops mein specific loop ko target karne ke liye.
- `for...in` arrays ke liye recommend nahi — traditional `for` ya `for...of` behtar.
- `for...of` ke sath destructuring: `for (const [key, val] of Object.entries(obj))`

✎ Coding Practice

1. Ek function `findFirstNegative(arr)` likho jo array mein loop kare aur pehla negative number milte hi `break` se return kare.
2. Ek function `sumPositives(arr)` likho jo negative numbers ko `continue` se skip kar ke sirf positives ka sum nikale.
3. Ek nested loop (2D grid, 5x5) banao jo labeled statement se ek specific target value milte hi dono loops break kare.

Real-World Use Case

1. Ek function `findFirstAvailableSeat(seats)` likho jo booking system jaisa array mein loop kare aur pehli available seat milte hi `break` kare.
   Hint / Kahan use hota hai: Real use: Ticket/seat booking systems (bus, cinema) isi linear search logic se available slot dhoondte hain.
2. Ek function `skipInvalidEntries(records)` likho jo `continue` se null/invalid records skip kar ke sirf valid records process kare.
   Hint / Kahan use hota hai: Real use: Data import/CSV upload features mein invalid rows isi tarah skip ki jati hain bina poora process rokay.

31 Conditionals — Aur Deep

- Truthy/Falsy: sirf `false, undefined, null, 0, NaN, ''` falsy hain, baaki sab truthy.
- Common mistake: `if (x === 5 || 7 || 10)` hamesha true hota hai — har condition poori likhni parti hai.
- `switch` — break bhoolna common bug hai (fall-through).
- Ternary koi bhi expression/function call ke andar chal sakta hai.

✎ Coding Practice

1. Ek function `isEmpty(value)` likho jo truthy/falsy check use kar ke batae ke value 'empty' hai (0, '', null, undefined, NaN sab ke liye true).
2. Ek function `checkNumber(x)` sahi tareeqe se likho jo check kare ke x, 5, 7, ya 10 mein se koi ek hai (galat wale `||` pattern ko avoid kar ke).
3. Ek `switch` based function `getSeason(month)` likho (1-12 input) jo season ka naam return kare — multiple cases ko intentionally group karo (fall-through use kar ke, jaise 12,1,2 → Winter).

Real-World Use Case

1. Ek function `isFormValid(formData)` likho jo truthy/falsy check se dekhe ke koi required field empty to nahi (bina explicit `=== ''` ke).

   Hint / Kahan use hota hai: Real use: Quick form validation checks truthy/falsy ka use karke concise likhi jati hain.

2. Ek `switch` based function `getOrderStatusLabel(statusCode)` banao jaisa real order tracking system mein hota hai (pending, shipped, delivered, cancelled).

   Hint / Kahan use hota hai: Real use: E-commerce order tracking UI mein status codes ko readable labels mein isi tarah convert kiya jata hai.

32 Arrays — Deep Dive (Methods & Internals)

- Arrays technically objects hain (reference type). `.length` = sabse bara numeric index + 1.
- Stack (push/pop, LIFO) vs Queue (push/shift, FIFO). push/pop fast, shift/unshift slow.
- `.splice(start, deleteCount, ...items)` — remove/insert. `.slice(start, end)` — copy (non-destructive, sirf references copy hote hain objects ke liye).
- `.at(-1)` — negative indexing (modern).
- Sorting: `.sort()` destructive hai. Searching: `.indexOf()`, `.findIndex()`, `.find()`.
- Existence: `.includes()`, `.some()`, `.every()`. Merge: `.concat()` ya spread. String convert: `.join()`.
- Arrays ko `==` / `===` se compare mat karo — loop ya `.every()` use karo.

✎ Coding Practice

1. Ek `Stack` class banao (push/pop se) aur ek `Queue` class banao (push/shift se), dono par 3-3 items test karo.
2. Ek array of `products` objects (name, price) banao aur ek function `findProductByName(products, name)` likho `.find()` use kar ke.
3. Ek function `arraysEqual(arr1, arr2)` likho jo do arrays ko element-by-element compare kar ke boolean return kare (bina `===` directly arrays par use kiye).

Real-World Use Case

1. Ek function `sortCoursesByPrice(courses)` likho jo array of course objects ko price ke hisaab se `.sort()` kare (low to high).
   Hint / Kahan use hota hai: Real use: 'Sort by price' filter jo har e-commerce/course platform mein hota hai isi comparator logic se kaam karta hai.
2. Ek function `searchStudentById(students, id)` likho jo `.find()` se specific student object return kare (ya undefined agar na mile).
   Hint / Kahan use hota hai: Real use: Admin dashboards mein ID se record dhoondna (edit/view profile) isi pattern se hota hai.

33 Functions — Deep Dive

- Parameter (declare waqt) vs Argument (call waqt) — common confusion.
- **ASI gotcha:** `return` aur value same line par honi chahiye, warna `undefined` return hoga.
- Function Declaration vs Function Expression. Callback functions — parameter ki tarah pass hone wala function.
- **IIFE:** function jo create hote hi turant khud call ho jaye — privacy ke liye use hota hai.
- Arrow function limitations: apni `this` binding nahi, constructor nahi ban sakti, `arguments` object nahi.

✎ Coding Practice

1. Ek function `getUserObject(id)` likho jisme ASI ki galti jaan-boojh kar karo (return aur object alag line par), phir usko fix karo.
2. Ek function `processArray(arr, callback)` likho jo har element par diya gaya callback function apply kare aur naya array return kare (apna custom `.map()`).
3. Ek IIFE likho jo turant execute ho kar ek private `counter` maintain kare aur `increment` / `getValue` functions return kare.

Real-World Use Case

1. Ek function `validateAndSubmit(formData, onSuccess)` likho jo validation pass hone par callback `onSuccess` ko call kare.

   Hint / Kahan use hota hai: Real use: Form submission handlers mein callback pattern se success ke baad specific action (redirect, toast) trigger hota hai.

2. Ek IIFE likho jo app start hote hi ek 'config' object (API base URL waghera) initialize kare aur privately rakhe.

   Hint / Kahan use hota hai: Real use: App configuration modules isi IIFE pattern se ek dafa initialize hote hain aur global namespace pollute nahi karte.

34 call(), apply(), aur bind() — this Ko Control Karna

- Teeno functions par available (functions bhi objects hain).
- `call(thisValue, arg1, arg2, ...)` — turant call, arguments comma-separated.
- `apply(thisValue, [argsArray])` — turant call, arguments array mein.
- `bind(thisValue, ...)` — naya function return karta hai (baad mein call karna parta hai). **Currying** ke liye use hota hai.
- Event listeners mein `this` ka masla — fix: constructor mein bind, ya arrow function wrap.

✎ Coding Practice

1. Do objects `person1` aur `person2` banao (dono mein name/age) — `person1` ka `introduce()` method `person2` ke context mein `.call()` se run karo.
2. Ek `calculateTotal(taxRate, price)` function likho aur `.bind()` se currying kar ke `calculateWithGST` (fixed 18% tax) function banao.
3. Ek function ko `.apply()` se call karo jisme arguments ek array `[a, b, c]` se pass hon — output `.call()` wale se compare karo.

Real-World Use Case

1. Ek shared `logger` object (jisme `log(message)` method ho jo `this.prefix` use kare) ko do alag services (`AuthService`, `PaymentService`) ke context mein `.call()` se use karo.

   Hint / Kahan use hota hai: Real use: Shared logging utilities ko multiple modules mein reuse karne ke liye isi 'borrowing' pattern ka use hota hai.

2. Ek `createApiClient(baseURL)` function banao jo `.bind()` se ek 'fixed base URL' wala fetch function return kare (currying).

   Hint / Kahan use hota hai: Real use: Alag-alag microservices ke liye base URL fix kar ke reusable API client functions isi tarah banaye jate hain.

35 Practice Exercises Reference (Objects & Arrays)

- Ek GitHub challenge set jo objects/arrays methods practice karne ke liye use ho sakta hai — `peeps` array aur `comments` object ke sath.
- **Easier:** count nikalna, full names array, `.every()` / `.some()`, `.filter()`, `.sort()`, `.map()`.
- **Harder:** comments list karna, filter, sort by rating, grouping, average nikalna — `reduce` jaisi patterns.

✎ Coding Practice

1. Ek `peeps` array (name, age) banao aur ek function likho jo `.every()` se check kare ke sab 18 se bare hain, aur ek jo `.some()` se check kare ke koi 60 se bara hai.
2. Usi `peeps` array ko age ke hisaab se `.sort()` se ascending order mein arrange karo (comparison function ke sath).
3. Ek `comments` array (text, rating 1-5) banao aur ek function `averageRating(comments)` likho jo `.reduce()` se average nikale.

Real-World Use Case

1. Ek `enrolledStudents` array (name, progress%) banao aur function likho jo `.filter()` se un students ko nikale jinka progress 100% hai (course complete).
   Hint / Kahan use hota hai: Real use: Course completion certificates isi filter logic se eligible students ko identify karte hain.
2. Ek `reviews` array (studentName, rating, comment) banao aur average course rating `.reduce()` se calculate karo jaisa course detail page dikhata hai.
   Hint / Kahan use hota hai: Real use: Udemy/Coursera jaisi platforms course rating badge isi calculation se dikhate hain.

36 this Keyword — Deep Dive

- `this` ki value **call se decide hoti hai**, definition se nahi (runtime binding).
- Standalone call: non-strict mein `this = window`, strict mein `this = undefined`.
- Arrow functions: lexical `this` — surrounding scope se leti hain, `call/apply/bind` override nahi kar sakte.
- Constructors: `this` naye object ko refer karta hai. Class instance methods: `this` = instance. Static methods: `this` = class.
- DOM event handlers: `this` = jis element par listener attach hua.

✎ Coding Practice

1. Ek object banao jisme ek regular function method aur ek standalone (usi function ko variable mein assign kar ke) call ho — dono ke `this` output print kar ke compare karo.
2. Ek `Counter` class banao jisme ek instance method `increment()` aur ek static method `Counter.describe()` ho — dikhao `this` dono mein kya refer karta hai.
3. HTML mein ek button banao jispar event listener lage — `console.log(this)` se dikhao ke `this` button element ko refer karta hai, phir usi listener ko arrow function se dobara likh kar farq dikhao.

Real-World Use Case

1. Ek `ApiService` object banao jisme `baseUrl` aur ek method `getFullUrl(endpoint)` ho jo `this.baseUrl` use kare — dikhao standalone call karne par `this` undefined ho jata hai.

   Hint / Kahan use hota hai: Real use: Ye ek common bug hai jab method ko object se destructure kar ke standalone use karte hain — API service classes isi issue se bachne ke liye arrow functions ya bind use karte hain.

2. Ek class-based `ThemeToggle` banao jisme ek button click listener class method ko call kare — dikhao `this` ka masla aur usko constructor mein `.bind(this)` se fix karo.

   Hint / Kahan use hota hai: Real use: React class components mein (purane codebases) yehi `this.method = this.method.bind(this)` pattern constructor mein likhna parta tha.

37 Asynchronous JavaScript — Callbacks, Promises, Async/Await

- Synchronous: top-se-bottom. Asynchronous: time-consuming tasks side mein bhej diye jate hain.
- `setTimeout(callback, ms)` — built-in async function.
- **Callback hell:** deeply nested async steps — mushkil se parhne wala code.
- **Promise:** states — pending, resolved/fulfilled, rejected. `.then()` chaining, `.catch()`, `.finally()`.
- **Async/Await:** cleaner, synchronous-jaisa syntax. `await` sirf `async` function ke andar. Error handling: `try/catch/finally`.

✎ Coding Practice

1. Ek function `delay(ms)` likho jo `ms` milliseconds baad resolve hone wali Promise return kare, aur usko `.then()` se test karo.
2. Ek function `fetchUser(id)` banao jo ek Promise return kare — agar `id > 0` ho to resolve, warna reject kare — `async/await` aur `try/catch` se call karo.
3. 3 alag delays wale Promises (`delay(1000)`, `delay(500)`, `delay(1500)`) banao aur `Promise.all()` se sabka wait karo, phir result print karo.

Real-World Use Case

1. Ek function `fetchCourseData(courseId)` banao jo Promise return kare (setTimeout se simulate) aur `async/await` se course title print kare.
   Hint / Kahan use hota hai: Real use: Tumhare Angular frontend mein Laravel API se course data isi async pattern se fetch hota hai.
2. Ek function `submitDonation(amount)` banao jo Promise return kare — agar amount 0 se kam ho to reject, warna resolve — `try/catch` se error handle karo.
   Hint / Kahan use hota hai: Real use: Charity module ke donation submission mein isi tarah validation + async API call combine hoti hai.

38 ES Modules — export / import

- Named exports: `export let x`, ya `export { name1, name2 };`. Default export: sirf ek per file.
- Named imports: `import { sayHi } from './say.js';`. Default import: `import User from './user.js';`. Import all: `import * as say`.
- Renaming: `as` keyword. Re-export: `export { sayHi } from './say.js';`
- Rules: import/export statements top level par hone chahiye.

✎ Coding Practice

1. Ek `mathUtils.js` file banao jisme `add` aur `subtract` functions named-export hon, aur ek `main.js` mein import kar ke use karo.
2. Ek `User.js` file banao jisme ek `class User` default-export ho, aur usko `main.js` mein import kar ke ek instance banao.
3. Ek `index.js` banao jo do alag files (`math.js`, `string.js`) se functions re-export kare — ek single entry point ki tarah.

Real-World Use Case

1. Ek `validators.js` module banao jisme `isValidEmail` aur `isValidPassword` functions named-export hon, aur `signup.js` mein import kar ke use karo.

   Hint / Kahan use hota hai: Real use: Form validation utilities har real project mein isi tarah alag module mein rakh kar reuse ki jati hain.

2. Ek `apiConfig.js` module banao jisme ek default-exported object ho (`{baseUrl, timeout}`), aur usko kisi `service.js` mein import karo.

   Hint / Kahan use hota hai: Real use: API configuration centralize karne ke liye ek hi config file poore app mein import hoti hai.

39 JSON (JavaScript Object Notation) — Deep Dive

- JSON JavaScript ka superset hai — lightweight, readable, APIs/config mein widely use hota hai.
- Supported types: strings, numbers, booleans, null, arrays, objects. Har key double quotes mein.
- `JSON.parse(jsonString)` — string se object. `JSON.stringify(jsObject)` — object se string.

✎ Coding Practice

1. Ek object banao, `JSON.stringify()` se string banao, phir `JSON.parse()` se wapas object banao — dono ki `typeof` print karo.
2. Ek function `deepClone(obj)` likho jo `JSON.parse(JSON.stringify(obj))` use kar ke ek deep copy banaye — test karo nested object ke sath.
3. Ek nested JSON string (array ke andar objects) hardcode karo aur `JSON.parse()` se parse kar ke usme se specific values nikalo.

Real-World Use Case

1. Ek fake API response string (JSON) hardcode karo jisme `{success: true, data: {students: [...]}}` ho, phir `JSON.parse()` se students array nikal kar loop karo.

   Hint / Kahan use hota hai: Real use: Har REST API response yehi structure follow karti hai — success flag + nested data — jise frontend parse kar ke use karta hai.

2. Ek `settings` object ko `localStorage` mein save karne se pehle `JSON.stringify()` karo, phir wapas load karte waqt `JSON.parse()` karo.

   Hint / Kahan use hota hai: Real use: Chunki localStorage sirf strings store karta hai, complex objects (user preferences) ko hamesha JSON stringify/parse karna parta hai.

40 DOM — Traversal & Events (Deep Dive)

- DOM tree: `document` root, elements/attributes/text/comments sab nodes hain.
- Traversal: `.parentNode` / `.parentElement`, `.childNodes` / `.children`, `.previousSibling` / `.nextElemen` waghera.
- Event propagation: **Capturing → Target → Bubbling**. `event.stopPropagation()`, `event.preventDefault()`.
- **Event delegation:** ek listener parent par, bubbling se child identify — kam code, dynamic elements bhi kaam karte hain.
- `.innerHTML` se XSS risk — user input ke liye `.textContent` prefer karo.

✎ Coding Practice

1. Ek `<ul>` mein 5 `<li>` banao — parent par ek hi event listener lagao jo event delegation se click hue `li` ka text console.log kare (`event.target` use kar ke).
2. Us list mein ek 'Add Item' button banao jo naya `<li>` dynamically add kare — dikhao ke event delegation naye add hue items par bhi kaam karta hai.
3. Nested `<div>` elements banao (parent aur child) — dono par click listeners lagao aur dikhao `event.stopPropagation()` ke sath aur bagair bubbling kaise differ karti hai.

Real-World Use Case

1. Ek product grid banao jisme har card par 'Add to Cart' button ho — event delegation se ek hi listener parent grid par lagao.

   Hint / Kahan use hota hai: Real use: Bare product listings (100+ items) mein performance ke liye individual listeners ki bajaye event delegation use hoti hai.

2. Ek dropdown menu banao jo bahar click karne par band ho jaye — `document` par listener lagao aur `event.target.closest()` se check karo click menu ke bahar hua.

   Hint / Kahan use hota hai: Real use: Har dropdown/modal component (navbar menus, profile dropdown) isi 'click outside to close' pattern se implement hota hai.

41 Modern JavaScript — Quick Reference (Pro Tips)

- Debugging: `console.log({variableName})`, `console.table()`, `console.time()` / `timeEnd()`, `console.trace()`.
- Object destructuring in function parameters: `function feed({ name, food }) {...}`
- Spread syntax: objects merge `{...obj1, ...obj2}`, arrays extend `[...arr, newItem]`.
- `Array.prototype.reduce()` — array ko single value mein accumulate karta hai.

✎ Coding Practice

1. Ek array of objects (5 products: name, price) banao aur `console.table()` se table format mein print karo.
2. Ek function `createProfile({name, age, city})` likho (destructured parameters ke sath) jo ek formatted string return kare.
3. Do config objects `defaults` aur `userSettings` banao — spread syntax se merge karo, phir `.reduce()` se kisi orders array ka grand total nikalo.

Real-World Use Case

1. Ek `orders` array (orderId, amount, status) ko `console.table()` se admin dashboard jaisa table format mein dikhao.

   Hint / Kahan use hota hai: Real use: Debugging ke waqt backend se aane wala data isi tarah table format mein inspect karna easy hota hai.

2. Spread syntax se `defaultUserSettings` aur `userOverrides` objects merge karo jaisa real settings/preferences system karta hai.

   Hint / Kahan use hota hai: Real use: User preference systems (notifications on/off, theme) mein defaults ko user-specific overrides ke sath merge karna common pattern hai.